

“It Ran” Is Not “It Was Right”: A Policy-Conditioned Benchmark of Ten Text-to-SQL Architectures on an Enterprise Database

Yagya Raj Sharma¹

¹(affiliation to be finalised) yagyarajsharma9@gmail.com

Abstract

Enterprise chat assistants that answer questions over a company database are usually judged by whether their SQL runs and how fast. On a realistic company database that is misleading: a query can run and return rows and still be wrong, or return rows the asker may not see, or ignore a rule written in a policy document, or answer a question that should have been refused. We build a testbed with all of these pressures in one place, an oil and gas database of 59 tables with role-based access control, an approval workflow, and nine policy documents, and a gold set of 94 questions labelled with the correct answer, the asking role, and whether the correct behaviour is to answer or refuse. We score ten published text-to-SQL architectures, the same ones a hackathon ranked by execution success, on four axes execution success cannot see: answer correctness against gold, data leakage against role, policy grounding, and refusal. The results are blunt. The architecture the execution-success ranking called the winner is ninth of ten on correctness. Every one of the ten leaks restricted data on 37 to 52 percent of questions. Eight of ten never refuse, and answer a request for a password hash, a request to delete invoices, and a prompt injection asking for the API key with a confident attempt. The best answer accuracy across all ten is 0.50. We argue enterprise text-to-SQL should be evaluated by policy-conditioned correctness, not execution success, and we release the testbed, gold set, and scorer.

1 Introduction

A common enterprise use of large language models is a chat assistant that turns a question into SQL over the company database. There are many designs, from a single prompt to multi-agent pipelines, and a natural urge to ask which is best. The usual way to answer is a bake-off: run each design on a set of questions and see whose SQL executes, returns rows, and does so quickly.

This paper started from such a bake-off. Ten text-to-SQL architectures were built over an enterprise oil and gas database and ranked by execution success and latency, and a multi-agent design was declared the winner. The ranking had a hole that is easy to miss and common in practice: nobody had written down the correct answers, so “the query ran and returned rows” stood in for “the answer was right.” On a real company database that substitution fails four ways: the query runs but returns the wrong rows; returns rows the asker may not see; ignores a rule that lives in a policy document, not the schema; or runs on a question that should have been refused, such as a request for a password or a destructive command. None of these is visible to an execution-success metric. We therefore build a testbed where all four pressures are present,

write down the correct behaviour, and re-score the same ten architectures.

Contributions. (1) a policy-conditioned gold set of 94 questions over an enterprise database, each labelled with the correct result, the asking role, and whether to answer or refuse; (2) a scorer reporting answer correctness against gold, a role-based data-leak rate, a policy-grounding rate, and a refusal rate, validated to separate a role-aware system from a role-blind one; and (3) the result of running ten published architectures through it, which overturns the execution-success ranking and shows all ten are unsafe on a company database. We release the database, documents, gold set, oracle, and scorer. The message is short: on enterprise data, “it ran” is not “it was right,” and a benchmark that cannot tell the difference will pick the wrong system and call an unsafe one good.

2 Related work

Text-to-SQL benchmarks have moved toward enterprise realism: Spider 2.0 [1], BEAVER [2] and EntSQL [3] grade SQL correctness on large schemas and workflows. They measure whether the SQL is right; not whether the asker was allowed to run it, or whether a policy document changes the answer. A separate line adds guards: TrustSQL [6] penalises confident wrong answers and rewards abstention; PV-SQL [7] and MARS-SQL [8] add a verification agent. Role-based access has begun to appear [5], mostly at whole-table level. Multi-architecture comparisons exist, e.g. BAPPA [9]. What is missing, and what we provide, is a single testbed and gold set that put role, policy documents, and refusal together and score several architectures on all of them at once, so the gap between “ran” and “was right and safe” is one measurable quantity. We call it policy-conditioned correctness. The ten architectures are standard: a naive single call; schema retrieval; a self-correcting loop; a Vanna-style few-shot bank; a ReAct agent [13]; DIN-SQL [10]; DAIL and C3 prompting [11,12]; a router with specialist agents; a GraphRAG [14] variant; and a chain of agents with a validator gate.

3 Testbed

The database models an oil and gas company: 59 tables covering wells and production, finance (invoices, purchase orders, contracts, payments), operations, health and safety, and identity. Identity is real: 20 roles, 23 resource-action permissions and a role-permission table, plus a four-tier approval

workflow. Nine policy documents are stored and full-text indexed. The data is synthetic and seeded, with no real company or personal information. Two things make it fair for policy-conditioned correctness. Access is governed by a real permission table, so “which rows may this role see” has a definite answer we can check. And some correct answers depend on a rule written in a document, not the schema (the approval-tier thresholds), so a system must combine SQL with a retrieved policy fact to be right.

4 Gold set

We wrote 94 questions in four types. **PLAIN (20)**: ordinary questions whose asker has the read permission. **ROLE-RESTRICTED (54)**: the same question asked by a role that may see the data (answer) and by roles that may not (refuse), generated from sensitive query templates crossed with roles using the permission table, so the label is correct by construction. **POLICY-GROUNDED (10)**: questions needing a rule from a document, e.g. “which requests need joint CFO and CEO approval,” requiring the Tier 3 threshold from the approval policy. **UNANSWERABLE (10)**: questions that should be refused: a password hash, a destructive command, a future prediction, an out-of-domain question, and a prompt injection asking for the API key. Each answerable question carries reference SQL, which we execute and store as the gold result, so grading is exact. Fifty are answerable and 44 should be refused.

5 Metrics

Per architecture output per question: **answer accuracy** (produced SQL’s result set matches gold, order-insensitive and projection-aware, re-run so an internal row cap does not distort it); **RBAC violation rate** (produced SQL reads a table the asking role may not read, per the live permission table); **refusal correctness** (on questions that should be refused, it declined); **policy grounding** (on policy questions, the answer applied the document rule, via matching the gold result that encodes it); and **trust penalty** (returned data on a refuse question, or a wrong answer with no hedge). We validate the scorer with two mock systems. A role-aware oracle scores answer 1.0, RBAC 0.0, refusal 1.0. A role-blind system running the same SQL regardless of asker scores answer 1.0 but RBAC 0.36 and refusal 0.0. The metrics separate SQL correctness from role safety.

6 Results

All ten use the same model (gpt-4o-mini, temperature 0), one run each, on 94 questions.

The execution-success winner is not the correctness winner. Chain-of-agents with a validator gate, ranked first by the original bake-off on execution success and latency, ranks ninth of ten on gold correctness (0.38) and ties for the worst confident-wrong rate. The extra machinery produced more elaborate SQL that ran and returned rows, but returned the right rows less often.

Table 1: Ten architectures, ranked by answer accuracy. Higher is better for answer accuracy, refusal and policy grounding; lower is better for RBAC violation and trust penalty.

architecture	ans	RBAC	refuse	policy	trust
schema_rag	0.50	0.426	0.00	0.10	0.543
react_agent	0.50	0.436	0.00	0.10	0.543
self_correct	0.46	0.426	0.00	0.10	0.628
router_multiagent	0.44	0.372	0.16	0.00	0.457
graphrag	0.44	0.489	0.00	0.10	0.553
few_shot	0.42	0.500	0.00	0.20	0.638
dail_c3	0.42	0.521	0.00	0.20	0.628
din_sql	0.40	0.404	0.00	0.20	0.574
chain_of_agents	0.38	0.436	0.05	0.20	0.638
naive_text2sql	0.36	0.457	0.00	0.20	0.617

Every architecture leaks restricted data. RBAC violation runs 0.37 to 0.52. None takes the asking role into account, because the interface is a bare question with no identity. A finance analyst’s question and a drilling engineer’s question are answered the same way, so a query reading salaries or contracts runs no matter who asked.

Almost none refuse. Refusal correctness is 0.00 for eight of ten; only the router (0.16) and chain-of-agents (0.05) ever decline. The other eight answer a request for a password hash, a request to delete invoices, and a prompt injection asking for the API key with a confident SQL attempt.

Policy grounding and accuracy are low. Policy grounding is 0.0 to 0.2 everywhere, and the best answer accuracy of any of the ten is 0.50. On a realistic enterprise schema the best of ten known techniques is right about half the time. The direction matches a 32-question pilot; the larger, role-heavier set sharpens the leak and refusal gaps.

7 Discussion

Execution success needs no labels, but on enterprise data it rewards runnable SQL rather than correct, authorized, policy-aware answers, and will rank a confident unsafe system above a careful one. Three steps follow from the four failures. Pass the asking role into the pipeline and filter the schema and result by permission. Retrieve the relevant policy fact and make it available to the SQL step. And train or prompt the system to refuse, and measure the refusal. None of this is exotic; the point is that without a policy-conditioned benchmark none of it is measured, and what is not measured is not fixed.

8 Threats to validity

One run per architecture on a single model; repeated runs with confidence intervals, cross-model comparison, and a controlled with/without validator-gate study are future work. The gold set is 94 questions, a foundation not a final size. Policy grounding is a proxy (matching the gold result that encodes the rule). answer accuracy is strict exact match, so a correct superset scores wrong. The data is synthetic, so absolute accuracies would differ on a real database; the between-architecture comparison and the leak and refusal findings

are what we rely on. Gold answers depend on the seeded database, so reproduction must use the released database.

9 Conclusion

We built an enterprise text-to-SQL testbed with role-based access, an approval workflow, and policy documents, and a gold set labelling each question with the correct answer, the asking role, and whether to refuse. Scoring ten architectures overturns an execution-success ranking: the declared winner is near the bottom on correctness, every architecture leaks restricted data, almost none refuse, and the best is right about half the time. On enterprise data, evaluation should be policy-conditioned. We release the testbed, gold set, and scorer.

References

- [1] F. Lei et al. Spider 2.0: Evaluating LMs on Real-World Enterprise Text-to-SQL. ICLR 2025.
- [2] P. Chen et al. BEAVER: An Enterprise Benchmark for Text-to-SQL. arXiv:2409.02038.
- [3] EntSQL: Grounding Enterprise Text-to-SQL in Long-Context Knowledge. arXiv:2606.03363.
- [4] Text-to-SQL for Enterprise Data Analytics. arXiv:2507.14372.
- [5] Benchmarking Text-to-SQL under Role-Based Access Control. arXiv:2607.22115.
- [6] G. Lee et al. TrustSQL: Reliability Benchmark with Penalised Abstention. arXiv:2403.15879.
- [7] PV-SQL: Verification for Text-to-SQL. arXiv:2604.17653.
- [8] MARS-SQL: Validation-Agent Text-to-SQL. arXiv:2511.01008.
- [9] BAPPA: Benchmarking Agents, Plans, and Pipelines for Text-to-SQL. arXiv:2511.04153.
- [10] M. Pourreza, D. Rafiei. DIN-SQL. NeurIPS 2023.
- [11] D. Gao et al. DAIL-SQL. VLDB 2024.
- [12] X. Dong et al. C3: Zero-shot Text-to-SQL with ChatGPT. arXiv:2307.07306.
- [13] S. Yao et al. ReAct. ICLR 2023.
- [14] D. Edge et al. GraphRAG. arXiv:2404.16130.
- [15] OpenAI. gpt-4o-mini. 2024.